Trabajo Práctico 1 – Continual Learning

I309 – Visión Artificial Avanzada
Universidad de San Andrés

Introducción

El aprendizaje continuo (*continual learning*) aborda el desafío de entrenar modelos de *deep learning* sobre una secuencia de tareas sin olvidar el conocimiento adquirido previamente. Cuando un modelo se entrena en una nueva tarea, su rendimiento en tareas anteriores suele degradarse drásticamente, fenómeno conocido como **olvido catastrófico** (*catastrophic forgetting*).

En este trabajo práctico recorrerán el espacio de soluciones al olvido catastrófico de forma progresiva: comenzarán con un baseline naive de *fine-tuning*, avanzarán hacia métodos de regularización de parámetros y destilación de conocimiento (*Knowledge Distillation*), y finalmente implementarán **Co^{2}L** (*Contrastive Continual Learning*), propuesto por Cha et al. en ICCV 2021. La idea central de Co^{2}L es que las representaciones aprendidas mediante objetivos contrastivos supervisados son más transferibles a tareas futuras que las aprendidas con entropía cruzada estándar.

Conjunto de Datos

CIFAR-10 contiene 60 000 imágenes de 32×32 píxeles en 10 clases (50 000 de entrenamiento, 10 000 de test). Para el escenario de aprendizaje continuo se construye **Seq-CIFAR-10** dividiendo las 10 clases en 5 tareas secuenciales, cada una con 2 clases:

Tarea	Clases	Imágenes (train)
1	airplane, automobile	10 000
2	bird, cat	10 000
3	deer, dog	10 000
4	frog, horse	10 000
5	ship, truck	10 000

El modelo se entrena secuencialmente: primero con las clases de la Tarea 1, luego Tarea 2, y así sucesivamente. **En ningún momento tiene acceso al dataset completo de tareas anteriores**, salvo por un pequeño buffer de replay (memoria) de tamaño fijo cuando el método lo requiera.

2.1. Escenarios de Evaluación

Se evalúan dos escenarios sobre Seq-CIFAR-10:

- **Class-Incremental Learning (Class-IL):** en test, el modelo debe clasificar entre las 10 clases sin conocer a qué tarea pertenece cada muestra. Es el escenario más desafiante.

- **Task-Incremental Learning (Task-IL):** en test, el modelo conoce la tarea (y por lo tanto el subconjunto de 2 clases). Solo debe elegir entre las clases de esa tarea.

Objetivo

El objetivo del TP es implementar y comparar distintos enfoques de aprendizaje continuo, reproduciendo finalmente los resultados de Co2L reportados en el paper original (arxiv.org/pdf/2106.14413) sobre Seq-CIFAR-10.

Etapas del Trabajo

El trabajo se estructura en cuatro etapas. Cada una construye sobre la anterior y sus resultados deben quedar reflejados en el informe.

4.1. Preparación del Dataset

- Implementar la división de CIFAR-10 en N tareas secuenciales (por defecto $N = 5$, dos clases por tarea).
- Definir los *data loaders* correspondientes a cada tarea, asegurando que el entrenamiento de la tarea t no tenga acceso a los datos de las tareas $1, \ldots, t-1$.
- Implementar el buffer de memoria (*replay buffer*) de tamaño fijo, con una política de selección a elección (e.g. reservoir sampling).

4.2. Pre-entrenamiento con Aprendizaje Contrastivo Supervisado

Antes de abordar el escenario continuo completo, deberán pre-entrenar el backbone usando **Supervised Contrastive Learning** (SupCon, Khosla et al. 2020) sobre la **Tarea 0** (primer par de clases), y luego adjuntar una cabeza de clasificación lineal para resolver dicha tarea.

- Implementar la pérdida contrastiva supervisada $\mathcal{L}_{\text{SupCon}}$.
- Entrenar el encoder/backbone con $\mathcal{L}_{\text{SupCon}}$ durante un número razonable de épocas sobre los datos de la Tarea 0.
- Agregar y entrenar una cabeza de clasificación lineal (congelando el backbone) para resolver la Tarea 0 con entropía cruzada estándar.
- **Visualizaciones requeridas:**
  - Evolución de la pérdida durante el pre-entrenamiento.
  - Proyección 2D de los embeddings del backbone (e.g. con t-SNE o UMAP) al inicio, a mitad del entrenamiento y al final, para observar la formación de clusters.
  - Accuracy de clasificación sobre la Tarea 0 luego de agregar la cabeza lineal.

4.3. Métodos de Aprendizaje Continuo

Usando el backbone pre-entrenado en la Etapa 1 como punto de partida, implementen los siguientes métodos de CL. Para cada método, entrenar secuencialmente las 5 tareas y registrar las métricas Class-IL y Task-IL **después de aprender cada tarea**.

4.3.1. Fine-Tuning Naive (Baseline)

Reentrenar el modelo completo con entropía cruzada en cada tarea nueva, sin ningún mecanismo para mitigar el olvido. Este método sirve como **cota inferior** de referencia.

4.3.2. Elastic Weight Consolidation (EWC)

Implementar EWC (Kirkpatrick et al. 2017) : añadir un término de penalización $\lambda \sum_i F_i(\theta_i - \theta_i^*)^2$ donde F_i es la diagonal de la matriz de información de Fisher.

4.3.3. Learning without Forgetting (LwF)

Implementar LwF (Li & Hoiem, 2018): al entrenar la tarea t, usar las predicciones del modelo anterior como *soft targets* sobre los datos de la nueva tarea para preservar el conocimiento de tareas previas mediante destilación de conocimiento.

4.3.4. Contrastive Continual Learning (Co^{2}L)

Implementar el método completo propuesto en el paper de referencia:

- **Supervised Contrastive Loss** con replay buffer para la nueva tarea.
- **Asymmetric Distillation Loss** para preservar la estructura de representaciones del modelo anterior.
- Gestión del replay buffer (memory update al final de cada tarea).
- El objetivo combinado es: $\mathcal{L} = \mathcal{L}_{\text{SupCon}} + \lambda\, \mathcal{L}_{\text{distill}}$

Se espera que este método alcance los resultados reportados en el paper (Class-IL $\approx$ 47 %–52 %, Task-IL $\approx$ 88 %–92 %).

[Opcional] - Método adicional a elección

Implementar un método de CL adicional a elección, ya sea uno visto en clase o no (e.g. iCaRL, L2P, SI, DER++, etc.). Incluir en el informe una descripción conceptual del método elegido, su relación con los métodos anteriores, y comparar sus resultados con los demás.

4.4. Comparación de Resultados

- Reportar en una tabla unificada las métricas finales de Class-IL y Task-IL para todos los métodos implementados.
- Graficar la **curva de accuracy** (Class-IL y Task-IL) en función del número de tareas aprendidas, para todos los métodos en el mismo gráfico.
- Graficar opcionalmente la **curva de forgetting** por tarea: cómo cae la accuracy sobre la Tarea k a medida que se aprenden las tareas $k+1, \ldots, N$.
- Analizar y discutir las diferencias observadas entre métodos en el informe.

Informe y Grupos de Trabajo

Deberán presentar un informe de **no más de 8 páginas** exponiendo las decisiones de diseño, implementación y resultados de cada etapa. El trabajo es **grupal, en grupos de dos personas**.

Formatos y Fechas de Entrega

El informe y el código fuente se entregarán juntos mediante el Campus Virtual en un archivo `.zip` denominado `apellido1_apellido2_tp1.zip`.

La estructura del directorio debe ser la siguiente:

```
castro_roca_tp1.zip/
    imgs/
    castro_roca_tp1_informe.pdf
    tp1.ipynb        (o el formato de código utilizado)
```

Pueden entregar el código en el formato que hayan utilizado (notebook `.ipynb`, scripts `.py`, etc.). Lo importante es que el código sea reproducible: incluir instrucciones claras para ejecutarlo.